

Lecture 11

Method of Weighted Residuals

In this lecture, we introduce the method of weighted residuals which provides the most general formulation for the finite element method. To begin, let's focus on the particular problem of steady heat diffusion in a rod. This problem can be modeled as a one-dimensional PDE for the temperature, T :

$$(kT_x)_x = -f, \quad (11.1)$$

where $k(x)$ thermal conductivity of the material and $f(x)$ is the heat source (per unit length), respectively. Note that both k and f could be functions of x . Also, let the physical domain for the problem be from $x = -L/2$ to $x = L/2$.

Example 11.1 (Steady heat diffusion) Suppose that the rod has a length of $L = 2$, the thermal conductivity is constant, $k = 1$, and the heat source, $f(x) = 50e^x$. Assume that the temperature at the ends of the rod are to be maintained at $T(\pm 1) = 100$. Equation (11.1) can be integrated twice to obtain:

$$\begin{aligned} (kT_x)_x &= -f, \\ T_{xx} &= -50e^x, \\ T_x &= -50e^x + a, \\ T &= -50e^x + ax + b. \end{aligned}$$

Now, applying boundary conditions so that $T(\pm 1) = 100$,

$$\begin{aligned} -50e^1 + a + b &= 100, \\ -50e^{-1} - a + b &= 100. \end{aligned}$$

This is a 2×2 system which can be solved for a and b ,

$$\begin{aligned} a &= 50 \sinh 1 \\ b &= 100 + 50 \cosh 1, \end{aligned}$$

where $\cosh y = (e^y + e^{-y})/2$ and $\sinh y = (e^y - e^{-y})/2$. Thus, the exact solution is,

$$T = -50e^x + 50x \sinh 1 + 100 + 50 \cosh 1. \quad (11.2)$$

A plot of this solution is shown in Figure 11.1.

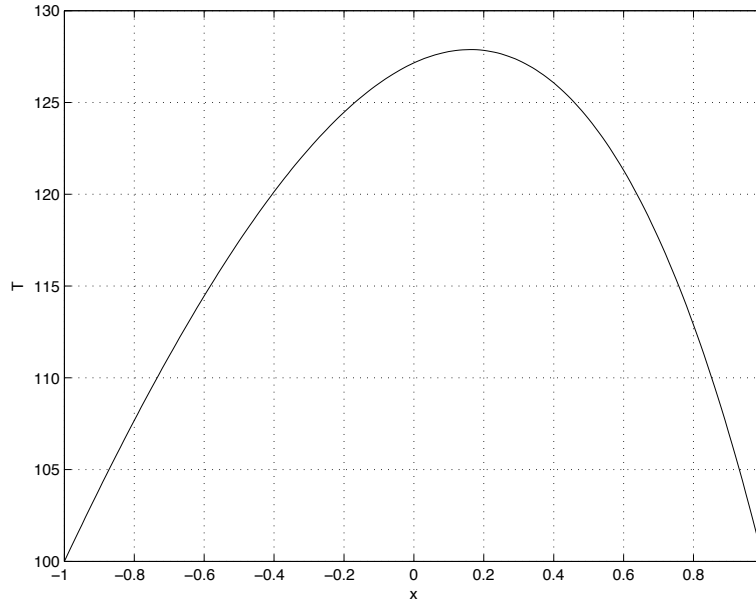


Figure 11.1: Temperature distribution for $f = 50e^x$, $L = 2$, and $k = 1$.

A common approach to approximating the solution to a PDE such as heat diffusion is to use a series of weighted functions. For example, for the temperature in Example 11.1 we might assume that,

$$\tilde{T}(x) = 100 + \sum_{j=1}^N a_j \phi_j(x),$$

where N is the number of terms (functions) in the approximation, $\phi_j(x)$ are the (known) functions, and a_j are the unknown function weights. The functions $\phi_j(x)$ are usually designed to satisfy the boundary conditions. So, in this example where the temperature is 100 at $x = \pm 1$, then $\phi_j(\pm 1) = 0$ (don't forget that $\tilde{T}$ was defined to include the constant term of 100).

The question remains what functions (and how many) to choose for $\phi_j(x)$. While many good choices exist, we will use polynomials in x because polynomial approximations are used extensively in finite element methods (our main interest). For this problem, the following ideas can be used to determine the form of the $\phi_j(x)$:

- First, we note that requiring $\phi_j(\pm 1) = 0$ places two conditions on each $\phi_j(x)$. These two conditions can be satisfied with a linear function of x , but the linear function which equals 0 at $x = \pm 1$ is simply 0. Since this does not add anything to the solution even after multiplying by a weight, the first non-trivial function would be a quadratic function.
- A quadratic function can be designed to satisfy the boundary conditions in the following manner,

$$\phi_1(x) = (1+x)(1-x).$$

By including factors which go to zero at the end points, we have constructed a quadratic function which will satisfy the required boundary conditions. A plot of $\phi_1(x)$ is shown in Figure 11.2.

- Suppose we wanted to include a cubic polynomial in the approximation, then one way we could do this is multiply $\phi_1(x)$ by x .

$$\phi_2(x) = x\phi_1(x) = x(1+x)(1-x).$$

Since $\phi_1(x)$ goes to zero at the end points, then so will $\phi_2(x)$. A plot of $\phi_2(x)$ is shown in Figure 11.2. There are actually some better ways to choose these higher-order polynomials than simply multiplying the lowest order polynomial by powers of x . The problem with the current approach is that if the number of terms were large (so that the powers of x would be large), then the set of polynomials (i.e. $\phi_j(x)$) become very poorly conditioned resulting in many numerical difficulties. We will not discuss issues of conditioning but more advanced texts on finite element methods or related subjects can be consulted. For low order polynomial approximations, the issues of conditioning do not play an important role.

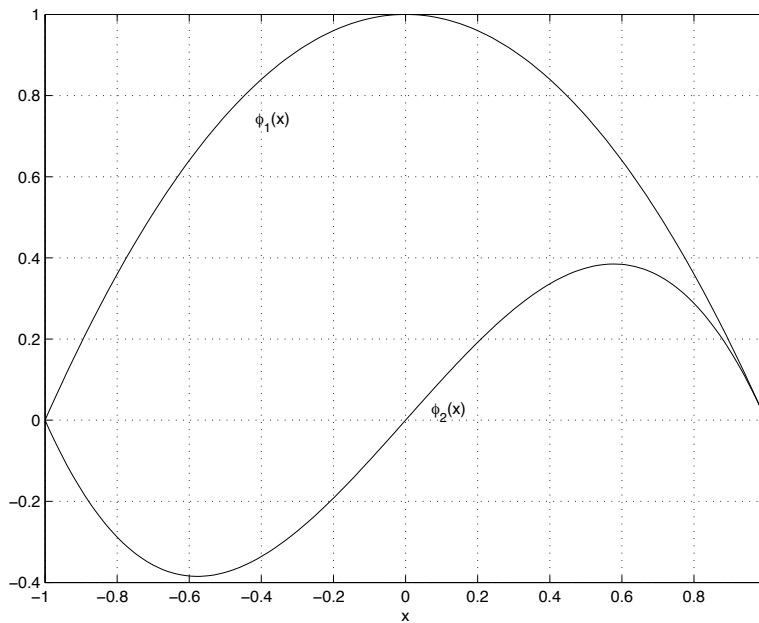


Figure 11.2: $\phi_1(x)$ and $\phi_2(x)$ for example heat diffusion problem.

Having selected a set of functions, we must now develop a way to determine values of a_j that will lead to a good approximation of the actual $T(x)$. While several ways exist to do this, we will focus on two methods in these introductory notes: the collocation method and the method of weighted residuals.

11.1 The Collocation Method

One approach to determine the N values of a_j would be to enforce the governing PDE at N points. Note that in general, the exact solution will not be a linear combination of the $\phi_j(x)$, so it will not be possible to enforce the PDE at every point in the domain. To see this, let's substitute $\tilde{T}(x)$ into Equation (11.1). Note that,

$$\begin{aligned}\tilde{T}_{xx} &= \frac{\partial^2}{\partial x^2} [a_1\phi_1(x) + a_2\phi_2(x)], \\ (\phi_1)_{xx} &= -2, \\ (\phi_2)_{xx} &= -6x, \\ \Rightarrow \tilde{T}_{xx} &= -2a_1 - 6a_2x.\end{aligned}$$

Next, we define a residual for Equation (11.1),

$$R(\tilde{T}, x) \equiv \left(k\tilde{T}_x\right)_x + f.$$

If the solution were exact, then the $R = 0$ for all x . Now, substitution of our chosen $\tilde{T}$ into the residual (recall $k = 1$ and $f = 50e^x$ in this example) gives,

$$R(\tilde{T}, x) = -2a_1 - 6a_2x + 50e^x. \quad (11.3)$$

Clearly, since a_1 and a_2 are constants (i.e. they do not depend on x), there is no way for this residual to be zero for all x .

The question remains, where should the N points be selected. The points at which the governing equation will be enforced are known as the collocation points. We will choose the relatively simple approach of equally subdividing the domain with $N = 2$ interior collocation points. For this domain from $-1 \leq x \leq 1$, the equi-distant collocation points would be at $x = \pm 1/3$. Thus, the two conditions for determining a_1 and a_2 are,

$$\begin{aligned}R(\tilde{T}, -1/3) &= 0, \\ R(\tilde{T}, 1/3) &= 0.\end{aligned}$$

From Equation (11.3) this gives,

$$\begin{aligned}-2a_1 + 2a_2 + 50e^{-1/3} &= 0, \\ -2a_1 - 2a_2 + 50e^{1/3} &= 0.\end{aligned}$$

Re-arranging this into a matrix form gives,

$$\begin{aligned}\begin{pmatrix} -2 & 2 \\ -2 & -2 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} &= \begin{pmatrix} -50e^{-1/3} \\ -50e^{1/3} \end{pmatrix}. \\ \Rightarrow \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} &= \begin{pmatrix} 25 \cosh 1/3 \\ 25 \sinh 1/3 \end{pmatrix} = \begin{pmatrix} 26.402 \\ 8.489 \end{pmatrix}.\end{aligned}$$

The results using this collocation method are shown in Figure 11.3 which includes plots of $\tilde{T}$, the error (i.e. $\tilde{T} - T$), and the residual. Note that the residual is clearly exactly zero at the collocation points (i.e. $x = \pm 1/3$), though the approximation is not exact at these points (i.e. $\tilde{T} \neq T$ at $x = \pm 1/3$).

11.2 The Method of Weighted Residuals

While the collocation method enforces the residual to be zero at N points, the method of weighted residuals requires N weighted integrals of the residual to be zero. A weighted residual is simply the integral of a weight function, $w(x)$ and the residual over the domain. For example, in the one-dimensional diffusion problem we are considering, a weighted residual is,

$$\int_{-1}^1 w(x) R(\tilde{T}, x) dx.$$

By choosing N weight functions, $w_i(x)$ for $1 \leq i \leq N$ and setting these N weighted residuals to zero, we may determine N values of a_j . We define the weighted residual for $w_i(x)$ to be,

$$R_i(\tilde{T}) \equiv \int_{-1}^1 w_i(x) R(\tilde{T}, x) dx.$$

And, the method of weighted residuals requires,

$$R_i(\tilde{T}) = 0 \quad \text{for } 1 \leq i \leq N.$$

In the method of weighted residuals, the next step is to determine appropriate weight functions. A common approach, known as the Galerkin method, is to set the weight functions equal to the functions used to approximate the solution. That is,

$$w_i(x) = \phi_i(x). \quad (\text{Galerkin}).$$

For the heat diffusion example we have been considering,

$$\begin{aligned} w_1(x) &= (1-x)(1+x), \\ w_2(x) &= x(1-x)(1+x). \end{aligned}$$

Now, we must calculate the weighted residuals. For the example,

$$\begin{aligned} R_1(\tilde{T}) &= \int_{-1}^1 w_1(x) R(\tilde{T}, x) dx, \\ &= \int_{-1}^1 (1-x)(1+x) (-2a_1 - 6a_2x + 50e^x) dx, \\ &= -\frac{8}{3}a_1 + 200e^{-1}. \end{aligned}$$

To do this integral, the following results were used,

$$\begin{aligned} \int_a^b (1-x)(1+x) dx &= \left[x - \frac{1}{3}x^3 \right]_a^b, \\ \int_a^b x(1-x)(1+x) dx &= \left[\frac{1}{2}x^2 - \frac{1}{4}x^4 \right]_a^b, \\ \int_a^b x^2 e^x dx &= [x^2 e^x - 2x e^x + 2e^x]_a^b. \end{aligned}$$

Similarly, calculating R_2 :

$$\begin{aligned} R_2(\tilde{T}) &= \int_{-1}^1 w_2(x) R(\tilde{T}, x) dx, \\ &= \int_{-1}^1 x(1-x)(1+x) (-2a_1 - 6a_2x + 50e^x) dx, \\ &= -\frac{8}{5}a_2 + 100e^1 - 700e^{-1}, \end{aligned}$$

where the following results have been used,

$$\begin{aligned} \int_a^b x^2(1-x)(1+x) dx &= \left[\frac{1}{3}x^3 - \frac{1}{5}x^5 \right]_a^b, \\ \int_a^b xe^x dx &= [xe^x - e^x]_a^b, \\ \int_a^b x^3e^x dx &= [x^3e^x - 3x^2e^x + 6xe^x - 6e^x]_a^b. \end{aligned}$$

Finally, we can solve for a_1 and a_2 by setting the weighted residuals R_1 and R_2 to zero,

$$\begin{aligned} -\frac{8}{3}a_1 + 200e^{-1} &= 0, \\ -\frac{8}{5}a_2 + 100e^1 - 700e^{-1} &= 0. \end{aligned}$$

This could be written as a 2×2 matrix equation and solved, but the equations are decoupled and can be readily solved,

$$\Rightarrow \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} \frac{125}{2}e^1 - \frac{875}{2}e^{-1} \\ 27.591 \\ 8.945 \end{pmatrix}.$$

The results using this method of weighted residuals are shown in Figure 11.4. Comparison with the collocation method results shows that the method of weighted residuals is more accurate in this case.

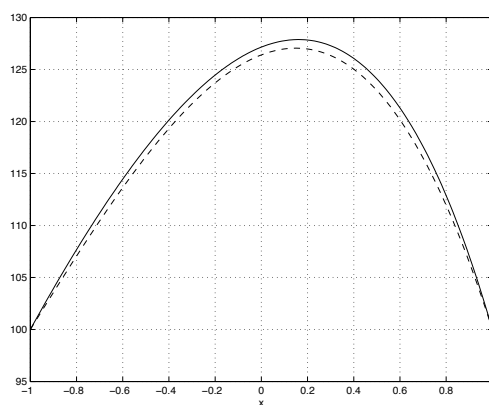
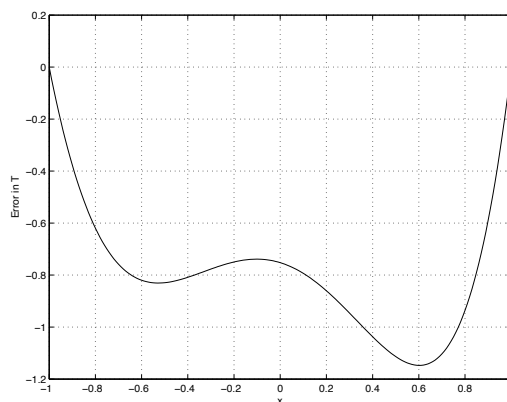
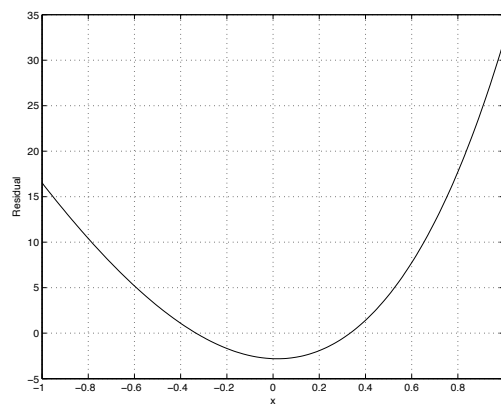
(a) Comparison of T (solid) and $\tilde{T}$ (dashed)(b) Error, $\tilde{T} - T$ (c) Residual, $R(\tilde{T}, x)$

Figure 11.3: Results for collocation method.

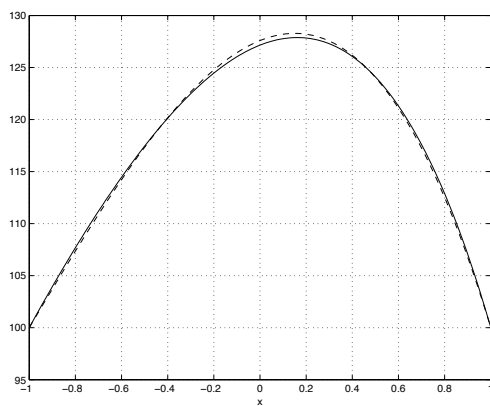
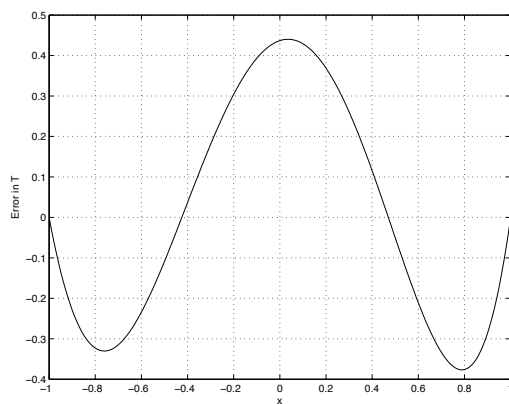
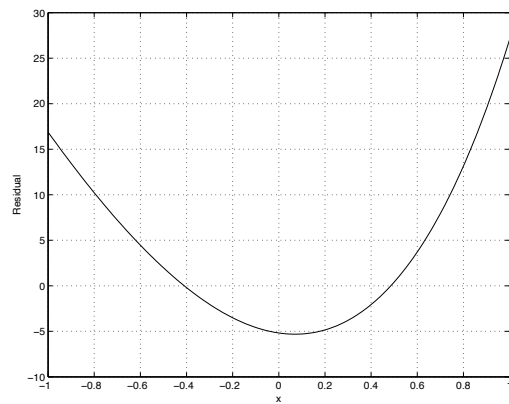
(a) Comparison of T (solid) and $\tilde{T}$ (dashed)(b) Error, $\tilde{T} - T$ (c) Residual, $R(\tilde{T}, x)$

Figure 11.4: Results for method of weighted residuals.

Lecture 12

The Finite Element Method for One-Dimensional Diffusion

12.1 Motivation

In this lecture, we introduce the finite element method (FEM). In its most general form, FEM is based on the method of weighted residuals. The application of the method of weighted residuals as described in Lecture 11 becomes difficult when the complexity of the problems increases, specifically in multiple dimensions and with varying properties (e.g. varying thermal conductivity in the heat diffusion problem). The heart of the difficulty in these applications is the construction of the weighted residual integrals. For the simple one-dimensional problem discussed in Lecture 11, these integrals were relatively easy to do. However, the analogous integrals in multiple dimensions with complex geometries are very difficult to evaluate without some additional form of numerical approximation. For example, consider the heat transfer problem shown in Figure 12.1 and imagine the difficulty in constructing a set of functions which satisfy the boundary conditions and then integrating the weighted residuals of these functions over the entire domain.

The finite element method offers one approach to approximating the solution and the weighted residual integrals in general situations and, therefore, makes possible the approximation of complex physical problems. The basic idea behind the finite element method is to discretize the domain into small cells (called elements in FEM) and use these elements to approximate the solution and evaluate the weighted residuals (an example mesh can be seen in Figure 12.2). Typically, in each element, the solution is approximated using polynomial functions. Then, the weighted residuals are evaluated on an element at a time and the resulting system of equations are solved to determine the coefficients on the polynomials in each element.

To introduce the basic concepts of the finite element method, we will first discuss the one-dimensional case. In future lectures, we will consider multiple dimensions and return to this complex heat transfer problem.

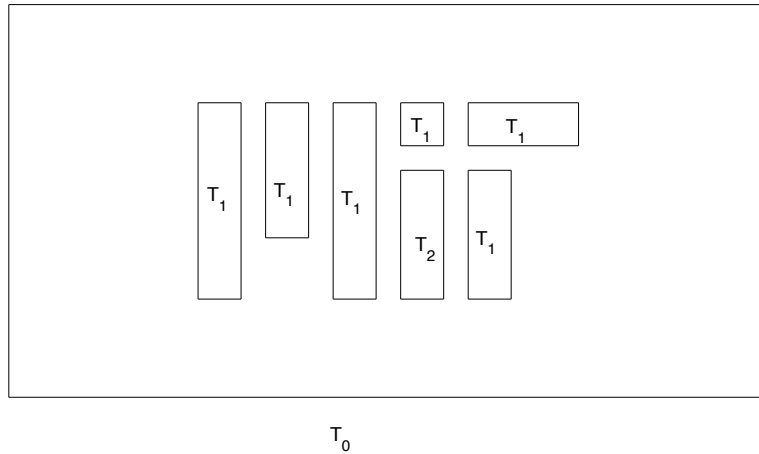


Figure 12.1: Heat transfer problem in a complex domain. Temperature at outer boundary is maintained at T_0 . Temperature of all of the internal rectangles except one are maintained at T_1 , while the remaining internal rectangle is maintained at T_2 .

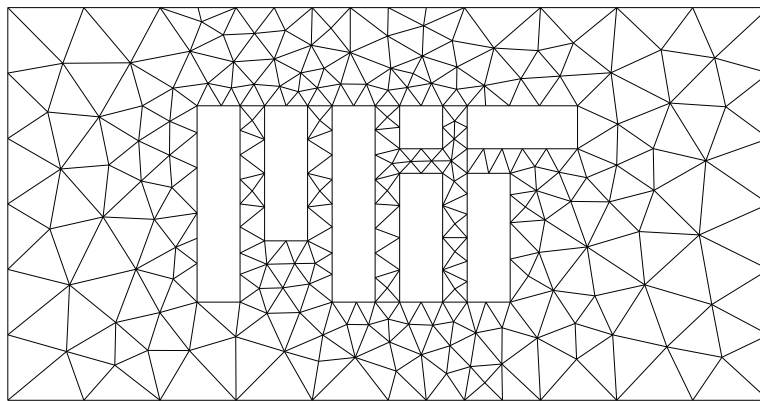


Figure 12.2: Mesh for heat transfer problem in a complex domain.

12.2 1-D Finite Element Mesh and Notation

Consider a mesh of one-dimensional elements as shown in Figure 12.3. As shown in the

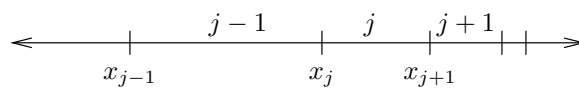


Figure 12.3: Mesh and notation for one-dimensional finite element method.

figure, element j is the region from $x_j \leq x \leq x_{j+1}$. Note, each element can have its own length,

$$\Delta x_j \equiv x_{j+1} - x_j.$$

12.3 1-D Linear Elements and the Nodal Basis

The finite element method typically uses polynomial functions inside each element. Furthermore, the approximation is usually required to be continuous from element to element. The simplest element which permits continuous functions would be to assume linear variations of x inside each element. This type of element is called a linear element (not too surprisingly). Using linear finite elements, a sample solution might look like that shown in Figure 12.4.

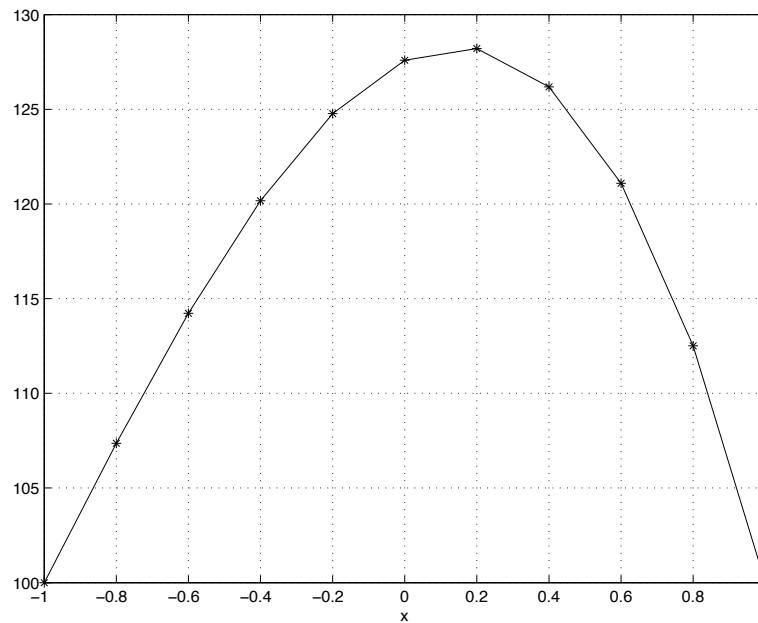


Figure 12.4: A linear element solution on a mesh with constant element size, $\Delta x_j = 0.2$.

A linear function can be described by two degrees of freedom. For example, a linear function within an element could be uniquely determined by the following combinations of information:

- the function values at the two endpoints (i.e. nodes) of the element,
- the function value at some point in the element and the function slope in the element.

For a mesh composed of N elements, a total of $N + 1$ degrees of freedom exist to describe a continuous, piecewise linear function. If the approximation were allowed to be discontinuous from element to element, then there would be $2N$ degrees of freedom but continuity removes $N - 1$ degrees of freedom (one per internal node). Thus, the degrees of freedom to describe a solution over a domain discretized into N linear elements could be (among other choices),

- the values of the function at the $N + 1$ nodes,
- the value of the function at some point in the domain and the slopes in the N elements.

The first choice is the most common and leads to what is known as the nodal basis.

We now derive the nodal basis functions (i.e. the $\phi_j(x)$) for linear elements. As discussed above, for N linear elements we have $N + 1$ degrees of freedom and therefore $N + 1$ basis functions, i.e.,

$$\tilde{T}(x) = \sum_{j=1}^{N+1} a_j \phi_j(x), \quad (12.1)$$

where a_j are the $N + 1$ degrees of freedom. For a nodal basis, we choose $a_j = \tilde{T}(x_j)$. Substituting this into Equation (12.1) gives,

$$\tilde{T}(x) = \sum_{j=1}^{N+1} \tilde{T}(x_j) \phi_j(x).$$

Now, evaluating this expansion at node i ,

$$\begin{aligned} \tilde{T}(x_i) &= \sum_{j=1}^{N+1} \tilde{T}(x_j) \phi_j(x_i), \\ \Rightarrow \phi_j(x_i) &= \begin{cases} 1, & \text{if } i = j, \\ 0, & \text{if } i \neq j. \end{cases} \end{aligned}$$

Finally, since the solutions vary linearly, this means that $\phi_j(x)$ is zero for $x < x_{j-1}$ and $x > x_{j+1}$, increases linearly from zero to one from x_{j-1} to x_j , and decreases linearly back to zero at x_{j+1} . $\phi_j(x)$ is shown in Figure 12.5. The, specific function is,

$$\phi_j(x) = \begin{cases} 0, & \text{for } x < x_{j-1}, \\ \frac{x - x_{j-1}}{\Delta x_{j-1}}, & \text{for } x_{j-1} < x < x_j, \\ \frac{x_{j+1} - x}{\Delta x_j}, & \text{for } x_j < x < x_{j+1}, \\ 0, & \text{for } x > x_{j+1}. \end{cases} \quad (12.2)$$

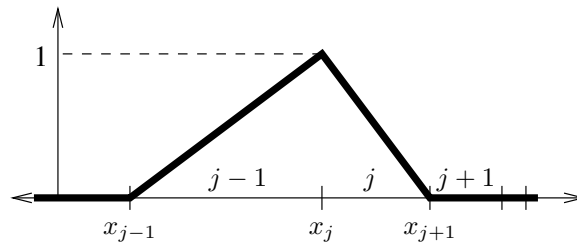


Figure 12.5: $\phi_j(x)$ for linear elements using a nodal basis.

12.4 1-D Diffusion Equation and Weighted Residual

In this lecture, we will concentrate on the one-dimensional, steady diffusion equation with a source term. As previously noted in Equation (11.1), the one-dimensional steady diffusion

equation with a source term is,

$$(kT_x)_x = -f,$$

where $k(x)$ thermal conductivity of the material and $f(x)$ is the heat source (per unit length), respectively. Note that both k and f could be functions of x . Also, let the physical domain for the problem be from $x = -L/2$ to $x = L/2$.

The residual corresponding to this equation is,

$$R(\tilde{T}, x) \equiv \left(k\tilde{T}_x \right)_x + f. \quad (12.3)$$

The weighted residual is defined as,

$$\int_{-L/2}^{L/2} w(x) R(\tilde{T}, x) dx = \int_{-L/2}^{L/2} w \left[\left(k\tilde{T}_x \right)_x + f \right] dx,$$

where $w(x)$ is an arbitrary weight function. In the finite element method, the weighted residual statement is usually integrated by parts for diffusion problems so that the number of derivatives on the weight function and the dependent variable ($\tilde{T}$) are equal. Thus, performing integration by parts gives the weighted residual as,

$$\left[w k\tilde{T}_x \right]_{-L/2}^{L/2} - \int_{-L/2}^{L/2} w_x k\tilde{T}_x dx + \int_{-L/2}^{L/2} w f dx.$$

The first term is on the boundaries of the domain and its use in setting boundary conditions is discussed in Section 12.6. The second and third terms are integrals over the entire domain.

Using the method of weighted residuals requires $N + 1$ weight functions. We will consider Galerkin methods in which the weight functions are chosen to be equal to the solution basis functions. Thus, following the method of weighted residuals, we define the i^{th} weighted residual as,

$$R_i(\tilde{T}) \equiv \left[\phi_i k\tilde{T}_x \right]_{-L/2}^{L/2} - \int_{-L/2}^{L/2} \phi_{ix} k\tilde{T}_x dx + \int_{-L/2}^{L/2} \phi_i f dx. \quad (12.4)$$

Example 12.1 (Evaluation of $\int_{-L/2}^{L/2} \phi_{ix} k\tilde{T}_x dx$ for constant k) While $\int_{-L/2}^{L/2} \phi_{ix} k\tilde{T}_x dx$ is a global integral (i.e. over the entire domain), in reality $\phi_i(x)$ is non-zero only over the two elements that include node i ,

$$\Rightarrow \int_{-L/2}^{L/2} \phi_{ix} k\tilde{T}_x dx = \int_{x_{i-1}}^{x_{i+1}} \phi_{ix} k\tilde{T}_x dx.$$

The derivative of basis function i is,

$$\phi_{ix}(x) = \begin{cases} 0, & \text{for } x < x_{i-1}, \\ \frac{1}{\Delta x_{i-1}}, & \text{for } x_{i-1} < x < x_i, \\ \frac{-1}{\Delta x_i}, & \text{for } x_i < x < x_{i+1}, \\ 0, & \text{for } x > x_{i+1}. \end{cases} \quad (12.5)$$

Thus,

$$\int_{x_{i-1}}^{x_{i+1}} \phi_{ix} k \tilde{T}_x dx = \frac{1}{\Delta x_{i-1}} \int_{x_{i-1}}^{x_i} k \tilde{T}_x dx - \frac{1}{\Delta x_i} \int_{x_i}^{x_{i+1}} k \tilde{T}_x dx.$$

Next, taking the derivative of $\tilde{T}$,

$$\tilde{T}_x(x) = \sum_{j=1}^{N+1} a_j \phi_{jx}(x),$$

For the integral in element $i-1$, the only non-zero contributions to $\tilde{T}_x$ are from ϕ_{i-1} and ϕ_i , specifically,

$$\text{In element } i-1: \quad \tilde{T}_x = a_{i-1} \phi_{i-1x} + a_i \phi_{ix} = \frac{a_i - a_{i-1}}{\Delta x_{i-1}}.$$

Similarly,

$$\text{In element } i: \quad \tilde{T}_x = a_i \phi_{ix} + a_{i+1} \phi_{i+1x} = \frac{a_{i+1} - a_i}{\Delta x_i}.$$

Substituting these expressions for the derivatives into the integral gives,

$$\int_{x_{i-1}}^{x_{i+1}} \phi_{ix} k \tilde{T}_x dx = \frac{a_i - a_{i-1}}{(\Delta x_{i-1})^2} \int_{x_{i-1}}^{x_i} k dx - \frac{a_{i+1} - a_i}{(\Delta x_i)^2} \int_{x_i}^{x_{i+1}} k dx.$$

At this point, we must integrate $k(x)$ in each element. Efficient numerical methods to approximate this integral are discussed in Section 12.5. For the situation in which k is constant throughout the domain the integral reduces to,

$$\text{For } k = \text{constant}: \quad \int_{x_{i-1}}^{x_{i+1}} \phi_{ix} k \tilde{T}_x dx = k \frac{a_i - a_{i-1}}{\Delta x_{i-1}} - k \frac{a_{i+1} - a_i}{\Delta x_i}.$$

This result should look somewhat familiar (hint: what does this formula reduce to when the element size is constant, $\Delta x_{i-1} = \Delta x_i$).

Example 12.2 (An FEM example for 1-D diffusion) The Matlab implementation of the finite element method for the problem described in Example 11.1 is shown below. Note, at the bottom of the script the exact solution and the error in the finite element solution are calculated and plotted. Interestingly, the FEM results for linear elements are exact at the nodes. However, in between the nodes (i.e. within the elements), there is error since a linear function is being used to represent a higher-order (curved) solution. The error, i.e. $\tilde{T}(x) - T(x)$, is shown in Figure 12.6(c) for both $N = 5$ and $N = 10$ solutions. A clear factor of four reduction is observed with the increased grid resolution leading to the conclusion that the method is second order accurate. Note: to construct this plot, each element was subdivided into 20 points and the FEM and exact solution were calculated at these points and compared.

```
% FEM solver for d2T/dx2 + f = 0 where f = 50 exp(x)
%
% BC's: T(-1) = 100 and T(1) = 100.
```

```

%
% Note: the finite element degrees of freedom are
%       stored in the vector, T.

% Number of elements
nElem = 10;
x = linspace(-1,1,nElem+1);

% Zero stiffness matrix
K = zeros(nElem+1, nElem+1);
F = zeros(nElem+1, 1);

% Loop over all elements and calculate stiffness and residuals
for elem = 1:nElem,

    n1 = elem;
    n2 = elem+1;

    x1 = x(n1);
    x2 = x(n2);

    dx = x2 - x1;

    % Add contribution to n1 weighted residual due to n1 function
    K(n1, n1) = K(n1, n1) - (1/dx);

    % Add contribution to n1 weighted residual due to n2 function
    K(n1, n2) = K(n1, n2) + (1/dx);

    % Add contribution to n2 weighted residual due to n1 function
    K(n2, n1) = K(n2, n1) + (1/dx);

    % Add contribution to n2 weighted residual due to n2 function
    K(n2, n2) = K(n2, n2) - (1/dx);

    % Add forcing term to n1 weighted residual
    F(n1) = F(n1) - (50*(exp(x2)-x2*exp(x1) + x1*exp(x1) - exp(x1))/dx);

    % Add forcing term to n2 weighted residual
    F(n2) = F(n2) - (50*(x2*exp(x2)-exp(x2)-x1*exp(x2)+exp(x1))/dx);

end

% Set Dirichlet conditions at x=-1

```

```
n1 = 1;
K(n1,:) = zeros(size(1,nElem+1));
K(n1, n1) = 1.0;
F(n1) = 100.0;

% Set Dirichlet conditions at x=1
n1 = nElem+1;
K(n1,:) = zeros(size(1,nElem+1));
K(n1, n1) = 1.0;
F(n1) = 100.0;

% Solve for solution
T = K\F;

% Plot solution
figure(1);
plot(x,T,'*-');
xlabel('x');
ylabel('T');

% For the exact solution, we need to use finer spacing to plot
% it correctly. If we only plot it at the nodes of the FEM mesh,
% the exact solution would also look linear between the nodes. To
% make sure there is always enough resolution relative to the FEM
% nodes, the size of the vector for plotting the exact solution is
% set to be 20 times the number of FEM nodes.
Npt = 20*nElem+1;
xe = linspace(-1,1,Npt);
Te = -50*exp(xe) + 50*xe*sinh(1) + 100 + 50*cosh(1);
hold on; plot(xe,Te); hold off;

% Plot the error. To do this, calculate the error on the same
% set of points in which the exact solution was plot. This
% requires that the location of the point xx(i) be found in the
% FEM mesh to construct the true solution at this point by linearly
% interpolating between the two nodes of the FEM mesh.

Terr(1) = T(1) - Te(1);
h = x(2)-x(1);
for i = 2:Npt-1,
    xxi = xe(i);
    Tei = Te(i);
```



```

j = floor((xxi-xe(1))/h) + 1;
x0 = x(j);
x1 = x(j+1);
T0 = T(j);
T1 = T(j+1);
xi = 2*(xxi - x0)/(x1-x0)-1; % This gives xi between +/-1
Ti = 0.5*(1-xi)*T0 + 0.5*(1+xi)*T1;
Terr(i) = Ti - Tei;
end
Terr(Npt) = T(nElem+1) - Te(Npt);

figure(2);
plot(xe,Terr);
xlabel('x');
ylabel('Error');

```

12.5 Gaussian Quadrature

The finite element method requires the calculation of integrals over individual elements, for example,

$$\int_{x_i}^{x_{i+1}} \phi_{ix} k \tilde{T}_x dx, \quad \text{or} \quad \int_{x_i}^{x_{i+1}} \phi_i f dx.$$

While in some settings these integrals can be calculated analytically, often they are too difficult. In this situation, numerical integration methods are used.

Gaussian quadrature is one of the most commonly applied numerical integration methods to this task. Gaussian integration approximates an integral as the weighted sum of the values of its integrand,

$$\int_{-1}^{+1} g(\xi) d\xi = \sum_{q=1}^{N_q} \alpha_q g(\xi_q), \quad (12.6)$$

where N_q is the number of quadrature points (i.e. integrand evaluations) and α_q and ξ_q are the weights and locations of integrand evaluations. Note that Gaussian quadrature rules are developed for specific integration limits, in this case between $\xi = -1$ to $+1$. Thus, for integration in an element, we will need to transform from x to ξ . See Example 12.3 for more on applying Gauss quadrature to finite element methods.

Gaussian quadrature integration rules are determined by requiring exact integration of polynomial integrands, i.e.,

$$g(\xi) = c_0 + c_1\xi + c_2\xi^2 + c_3\xi^3 + \cdots + c_M\xi^M,$$

for all values of c_m . Note that,

$$\int_{-1}^{+1} \xi^m d\xi = \begin{cases} 0 & \text{if } m = \text{odd} \\ \frac{2}{m+1} & \text{if } m = \text{even} \end{cases}$$

$$\Rightarrow \int_{-1}^{+1} \sum_{m=0}^M c_m \xi^m d\xi = 2 \left(c_0 + \frac{1}{3}c_2 + \frac{1}{5}c_4 + \cdots \right). \quad (12.7)$$

12.5.1 $N_q = 1$ Quadrature Rule

For $N_q = 1$, the quadrature rule is,

$$\int_{-1}^{+1} g(\xi) d\xi \approx \alpha_1 g(\xi_1).$$

Now, using Equation (12.7), we determine the highest order polynomial that is integrable by a single quadrature point,

$$\begin{aligned} 2 \left(c_0 + \frac{1}{3}c_2 + \frac{1}{5}c_4 + \cdots \right) &= \alpha_1 g(\xi_1) \\ &= \alpha_1 (c_0 + c_1 \xi_1 + c_2 \xi_1^2 + c_3 \xi_1^3 + \cdots + c_M \xi_1^M). \end{aligned}$$

Matching term by term gives,

$$\begin{aligned} c_0 &: 2 = \alpha_1 &\Rightarrow \alpha_1 = 2, \\ c_1 &: 0 = \alpha_1 \xi_1 &\Rightarrow \xi_1 = 0. \end{aligned}$$

Then, checking the c_2 term shows that it is not integrating exactly. So, with one point, a linear polynomial is the highest order polynomial that can be evaluated exactly and the quadrature rule is,

$$\int_{-1}^{+1} g(\xi) d\xi \approx \alpha_1 g(\xi_1), \quad \alpha_1 = 2, \quad \xi_1 = 0.$$

12.5.2 $N_q = 2$ Quadrature Rule

For $N_q = 2$, the quadrature rule is,

$$\int_{-1}^{+1} g(\xi) d\xi \approx \alpha_1 g(\xi_1) + \alpha_2 g(\xi_2).$$

Again using Equation (12.7), we determine the highest order polynomial that is integrable by two quadrature points,

$$\begin{aligned} 2 \left(c_0 + \frac{1}{3}c_2 + \frac{1}{5}c_4 + \cdots \right) &= \alpha_1 g(\xi_1) + \alpha_2 g(\xi_2) \\ &= \alpha_1 (c_0 + c_1 \xi_1 + c_2 \xi_1^2 + c_3 \xi_1^3 + \cdots + c_M \xi_1^M) + \\ &\quad \alpha_2 (c_0 + c_1 \xi_2 + c_2 \xi_2^2 + c_3 \xi_2^3 + \cdots + c_M \xi_2^M). \end{aligned}$$

Matching the first four terms gives the following constraints,

$$\begin{aligned} c_0 &: 2 = \alpha_1 + \alpha_2, \\ c_1 &: 0 = \alpha_1 \xi_1 + \alpha_2 \xi_2, \\ c_2 &: \frac{2}{3} = \alpha_1 \xi_1^2 + \alpha_2 \xi_2^2, \\ c_3 &: 0 = \alpha_1 \xi_1^3 + \alpha_2 \xi_2^3. \end{aligned}$$

These constraints can be met with,

$$\alpha_1 = \alpha_2 = 1, \quad \xi_1 = -\frac{1}{\sqrt{3}}, \quad \xi_2 = \frac{1}{\sqrt{3}}.$$

Thus, this rule will integrate cubic polynomials exactly.

Example 12.3 (Calculation of Forcing Integral with Gauss Quadrature) *We wish to apply Gaussian quadrature to evaluate the forcing integral,*

$$\int_{x_i}^{x_{i+1}} \phi_i f dx. \quad (12.8)$$

To do this, we first transform between the x and ξ space. For this integral in element i , the transformation would be,

$$x(\xi) = x_i + \frac{1}{2}(1 + \xi)(x_{i+1} - x_i), \quad \Rightarrow dx = \frac{1}{2}(x_{i+1} - x_i)d\xi.$$

Substitution in the forcing integral gives,

$$\int_{x_i}^{x_{i+1}} \phi_i f dx = \int_{-1}^{+1} \frac{1}{2}(x_{i+1} - x_i) \phi_i f d\xi, \quad \Rightarrow \quad g(\xi) = \frac{1}{2}(x_{i+1} - x_i) \phi_i[x(\xi)] f[x(\xi)].$$

Note that the dependence of ϕ_i and f on ξ is shown through the dependence of these functions on $x = x(\xi)$. However, for the basis functions, it is often easier to directly determine ϕ_i from ξ . For example, in the case of linear polynomial basis functions, the basis functions with element i can be written as,

$$\begin{aligned} \phi_1(\xi) &= \frac{1}{2}(1 - \xi), \\ \phi_2(\xi) &= \frac{1}{2}(1 + \xi). \end{aligned}$$

Clearly, these functions vary linearly with ξ . $\phi_1(\xi)$ is one at $\xi = -1$ and decreases linearly to zero at $\xi = +1$. And, vice-versa for $\phi_2(\xi)$. For the integral, we are considering in this example, $\phi_i(x)$ is equivalent to $\phi_i(\xi)$.

The following is a Matlab script that uses Gaussian quadrature to evaluate the forcing integral and solve the problem described in Example 11.1. The number of points being used is set at the beginning of the script. Results for both 1-point and 2-point quadrature are shown in Figures 12.7 and 12.8 for 5 and 10 elements. While the 2-point quadrature rule has lower error than the 1-point rule, both appear to be second-order accurate since the errors reduce by nearly a factor of 4 for the factor 2 change in mesh size. Also, the results are no longer exact at the nodes like they were in Example 12.2 (though the 2-point quadrature rules are quite close).

```
% FEM solver for d2T/dx2 + f = 0 where f = 50 exp(x)
%
% BC's: T(-1) = 100 and T(1) = 100.
```

```

%
% Gaussian quadrature is used in evaluating the forcing integral.
%
% Note: the finite element degrees of freedom are
%       stored in the vector, T.

% Number of elements
nElem = 5;
x = linspace(-1,1,nElem+1);

% Set quadrature rule
Nq = 2;
if (Nq == 1),
    alphaq(1) = 2.0; xiq(1) = 0.0;
elseif (Nq == 2),
    alphaq(1) = 1.0; xiq(1) = -1/sqrt(3);
    alphaq(2) = 1.0; xiq(2) = 1/sqrt(3);
else
    fprintf('Error: Unknown quadrature rule (Nq = %i)\n',Nq);
    return;
end

% Zero stiffness matrix
K = zeros(nElem+1, nElem+1);
F = zeros(nElem+1, 1);

% Loop over all elements and calculate stiffness and residuals
for elem = 1:nElem,

    n1 = elem;
    n2 = elem+1;

    x1 = x(n1);
    x2 = x(n2);

    dx = x2 - x1;

    % Add contribution to n1 weighted residual due to n1 function
    K(n1, n1) = K(n1, n1) - (1/dx);

    % Add contribution to n1 weighted residual due to n2 function
    K(n1, n2) = K(n1, n2) + (1/dx);

    % Add contribution to n2 weighted residual due to n1 function
    K(n2, n1) = K(n2, n1) + (1/dx);

```

```

% Add contribution to n2 weighted residual due to n2 function
K(n2, n2) = K(n2, n2) - (1/dx);

% Evaluate forcing term using quadrature
for nn = 1:Nq,

    % Get xi location of quadrature point
    xi = xiq(nn);

    % Calculate x location of quadrature point
    xq = x1 + 0.5*(1+xi)*dx;

    % Calculate f
    f = 50*exp(xq);

    % Calculate phi1 and phi2
    phi1 = 0.5*(1-xi);
    phi2 = 0.5*(1+xi);

    % Add forcing term to n1 weighted residual
    F(n1) = F(n1) - alphaq(nn)*0.5*phi1*f*dx;

    % Add forcing term to n2 weighted residual
    F(n2) = F(n2) - alphaq(nn)*0.5*phi2*f*dx;

end

end

% Set Dirichlet conditions at x=-1
n1 = 1;
K(n1,:) = zeros(size(1,nElem+1));
K(n1, n1) = 1.0;
F(n1) = 100.0;

% Set Dirichlet conditions at x=1
n1 = nElem+1;
K(n1,:) = zeros(size(1,nElem+1));
K(n1, n1) = 1.0;
F(n1) = 100.0;

% Solve for solution
T = K\F;

```

```

% Plot solution
figure(1);
plot(x,T,'*-');
xlabel('x');
ylabel('T');

% For the exact solution, we need to use finer spacing to plot
% it correctly. If we only plot it at the nodes of the FEM mesh,
% the exact solution would also look linear between the nodes. To
% make sure there is always enough resolution relative to the FEM
% nodes, the size of the vector for plotting the exact solution is
% set to be 20 times the number of FEM nodes.
Npt = 20*nElem+1;
xe = linspace(-1,1,Npt);
Te = -50*exp(xe) + 50*xe*sinh(1) + 100 + 50*cosh(1);
hold on; plot(xe,Te); hold off;

% Plot the error. To do this, calculate the error on the same
% set of points in which the exact solution was plot. This
% requires that the location of the point xx(i) be found in the
% FEM mesh to construct the true solution at this point by linearly
% interpolating between the two nodes of the FEM mesh.

Terr(1) = T(1) - Te(1);
h = x(2)-x(1);
for i = 2:Npt-1,
    xxi = xe(i);
    Tei = Te(i);
    j = floor((xxi-xe(1))/h) + 1;
    x0 = x(j);
    x1 = x(j+1);
    T0 = T(j);
    T1 = T(j+1);
    xi = 2*(xxi - x0)/(x1-x0)-1; % This gives xi between +/-1
    Ti = 0.5*(1-xi)*T0 + 0.5*(1+xi)*T1;
    Terr(i) = Ti - Tei;
end
Terr(Npt) = T(nElem+1) - Te(Npt);

figure(2);
plot(xe,Terr);
xlabel('x');
ylabel('Error');

```

12.6 Boundary Conditions

Boundary conditions generally fall into one of three types:

- Set $\tilde{T}$ at the boundary (known as a Dirichlet boundary condition). For heat transfer problems, this type of boundary condition occurs when the temperature is known at some portion of the boundary.
- Set $\tilde{T}_x$ at the boundary (known as a Neumann boundary condition). For heat transfer problems, this type of boundary condition occurs when the heat transfer rate is known at the boundary. For example, an adiabatic boundary would require that $\tilde{T}_x = 0$.
- Set $\alpha_0 \tilde{T} + \alpha_1 \tilde{T}_x$ at the boundary (known as a Robin boundary condition) where α_0 and α_1 do not depend on the temperature. For heat transfer problems, this type of boundary condition occurs when modeling convection (see Example 12.4).

Example 12.4 (Convection boundary condition for heat transfer) *Consider the flow of air over a solid object (see Figure 12.9) with the velocity and temperature of the air being, U_{ext} and T_{ext} , respectively. Aligning the x direction into the surface, then the heat transfer rate at the surface is,*

$$q_{wall} = -kT_x, \quad (12.9)$$

where $T(x)$ is the temperature inside the solid and k is the thermal conductivity of the solid. A common approach to modeling the heat transfer into the solid as a result of the airflow is based on specifying a heat transfer coefficient for the airflow, h_{ext} , which is related to the heat transfer rate by,

$$h_{ext} \equiv \frac{q_{wall}}{T_{ext} - T_{wall}}. \quad (12.10)$$

Note that h_{ext} is generally a function of the external velocity and other flow properties. Combining Equations (12.9) and (12.10) gives the following boundary condition at the surface of the solid,

$$-kT_x = h_{ext}(T_{ext} - T_{wall}), \quad (12.11)$$

where T_{wall} is the temperature of the solid at its surface. This equation can be re-arranged into the Robin boundary condition form,

$$-kT_x + h_{ext}T_{wall} = h_{ext}T_{ext}.$$

12.6.1 Implementation of Dirichlet Boundary Conditions

To demonstrate the implementation of a Dirichlet boundary condition, suppose that the value of the temperature is known at $x = -L/2$, specifically,

$$T(-L/2) = T_{left}.$$

This condition is set by forcing the corresponding nodal degree of freedom to be the desired value. At $x = -L/2$, the corresponding nodal degree of freedom would be a_1 (the value of the temperature at the first node), thus, the boundary condition is implemented as,

$$a_1 = T_{left}.$$

12.6.2 Implementation of Neumann Boundary Conditions

To demonstrate the implementation of a Neumann boundary condition, suppose that the heat transfer rate were known at $x = L/2$, specifically,

$$-kT_x(L/2) = q_{right}.$$

This condition is enforced through the weighted residual for the last node, $i = N + 1$. Specifically,

$$R_{N+1} = \left[\phi_{N+1} k \tilde{T}_x \right]_{-L/2}^{L/2} - \int_{-L/2}^{L/2} \phi_{N+1} k \tilde{T}_x dx + \int_{-L/2}^{L/2} \phi_{N+1} f dx.$$

Because $\phi_{N+1}(x)$ is zero except in the last element, this weighted residual reduces to,

$$R_{N+1} = \left(\phi_{N+1} k \tilde{T}_x \right) \Big|_{x=x_{N+1}} - \int_{x_N}^{x_{N+1}} \phi_{N+1} k \tilde{T}_x dx + \int_{x_N}^{x_{N+1}} \phi_{N+1} f dx.$$

The two integral terms are calculated in the standard manner. The first term is where the Neumann boundary condition is set through substitution of $-k\tilde{T}_x = q_{right}$. Specifically, the weighted residual becomes,

$$R_{N+1} = -\phi_{N+1}(x_{N+1})q_{right} - \int_{x_N}^{x_{N+1}} \phi_{N+1} k \tilde{T}_x dx + \int_{x_N}^{x_{N+1}} \phi_{N+1} f dx.$$

Note that the boundary term, $-\phi_{N+1}(x_{N+1})q_{right}$ does not depend on the temperature and thus this boundary condition does not impact the stiffness matrix.

12.6.3 Implementation of Robin Boundary Conditions

To demonstrate the implementation of a Robin boundary condition, suppose that a convective heat transfer boundary condition were to be set at $x = L/2$, specifically,

$$-kT_x(L/2) = h_{ext} [T_{ext} - T(L/2)].$$

For more on convective boundary conditions, see Example 12.4. Following the basic process outlined in the Neumann boundary condition, the weighted residual for $i = N + 1$ is,

$$R_{N+1} = \left(\phi_{N+1} k \tilde{T}_x \right) \Big|_{x=x_{N+1}} - \int_{x_N}^{x_{N+1}} \phi_{N+1} k \tilde{T}_x dx + \int_{x_N}^{x_{N+1}} \phi_{N+1} f dx.$$

Substituting $-k\tilde{T}_x = h_{ext} [T_{ext} - \tilde{T}(x_{N+1})]$ in the boundary term gives,

$$R_{N+1} = -\phi_{N+1}(x_{N+1})h_{ext} [T_{ext} - \tilde{T}(x_{N+1})] - \int_{x_N}^{x_{N+1}} \phi_{N+1} k \tilde{T}_x dx + \int_{x_N}^{x_{N+1}} \phi_{N+1} f dx.$$

As opposed to the Neumann boundary condition, the Robin boundary condition implementation does introduce a new dependence on the solution, specifically on $\tilde{T}(x_{N+1})$. This will cause a change in the stiffness matrix. Furthermore, the T_{ext} term will alter the right-hand side vector in the FEM numerical implementation.

12.6.4 Boundary Condition Implementation Details

In class, we will discuss the details of the implementation of the boundary conditions into a computer program using the following Matlab script.

```
% FEM solver for  $k \frac{d^2T}{dx^2} + f = 0$  where  $f = 50 \exp(x)$ 
%
% Thermal conductivity is set to one,  $k=1$ .
%
% BC's:
%
% At  $x=-1$ :  $T(-1) = 100$  (Dirichlet)
%
% At  $x=1$ , two options exist:
%
%           Specified heat transfer:  $dT/dX = q_{right}$ 
%
%           Convection:  $-k \frac{dT}{dx} = h_{ext} * (T_{ext} - T(1))$ 
%
% The choice of which bc to use is made through RightBC.
% If RightBC = 0, heat transfer rate is specified. Otherwise,
% a convection BC is applied.
%
% Gaussian quadrature is used in evaluating the forcing integral.
%
% Note: the finite element degrees of freedom are
%       stored in the vector, T.

clear all;

% Number of elements
nElem = 5;
x = linspace(-1,1,nElem+1);

% Set RightBC info
RightBC = 1;
if (RightBC == 0),
    qright = 0;
else,
    hext = 10;
    Text = 100;
end

% Set quadrature rule
Nq = 2;
```

```

if (Nq == 1),
    alphaq(1) = 2.0; xiq(1) = 0.0;
elseif (Nq == 2),
    alphaq(1) = 1.0; xiq(1) = -1/sqrt(3);
    alphaq(2) = 1.0; xiq(2) = 1/sqrt(3);
else
    fprintf('Error: Unknown quadrature rule (Nq = %i)\n',Nq);
    return;
end

% Zero stiffness matrix
K = zeros(nElem+1, nElem+1);
F = zeros(nElem+1, 1);

% Loop over all elements and calculate stiffness and residuals
for elem = 1:nElem,

    n1 = elem;
    n2 = elem+1;

    x1 = x(n1);
    x2 = x(n2);

    dx = x2 - x1;

    % Add contribution to n1 weighted residual due to n1 function
    K(n1, n1) = K(n1, n1) - (1/dx);

    % Add contribution to n1 weighted residual due to n2 function
    K(n1, n2) = K(n1, n2) + (1/dx);

    % Add contribution to n2 weighted residual due to n1 function
    K(n2, n1) = K(n2, n1) + (1/dx);

    % Add contribution to n2 weighted residual due to n2 function
    K(n2, n2) = K(n2, n2) - (1/dx);

    % Evaluate forcing term using quadrature
    for nn = 1:Nq,

        % Get xi location of quadrature point
        xi = xiq(nn);

        % Calculate x location of quadrature point
        xq = x1 + 0.5*(1+xi)*dx;

```

```

    % Calculate f
    f = 50*exp(xq);

    % Calculate phi1 and phi2
    phi1 = 0.5*(1-xi);
    phi2 = 0.5*(1+xi);

    % Add forcing term to n1 weighted residual
    F(n1) = F(n1) - alphaq(nn)*0.5*phi1*f*dx;

    % Add forcing term to n2 weighted residual
    F(n2) = F(n2) - alphaq(nn)*0.5*phi2*f*dx;

end

end

% Set Dirichlet conditions at x=-1
n1 = 1;
K(n1,:) = zeros(size(1,nElem+1));
K(n1, n1) = 1.0;
F(n1) = 100.0;

% Set boundary condition at x=1
n1 = nElem+1;
if (RightBC == 0), % Specify heat transfer rate (Neumann)

    F(n1) = F(n1) + qright;

else, % Convective

    K(n1,n1) = K(n1,n1) + hext;
    F(n1) = F(n1) + hext*Text;

end

% Solve for solution
T = K\F;

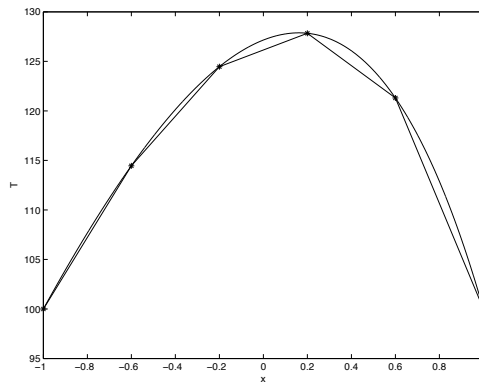
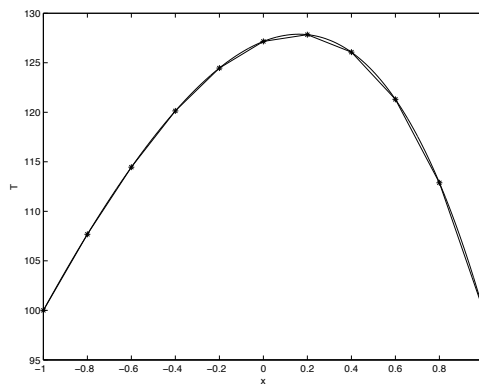
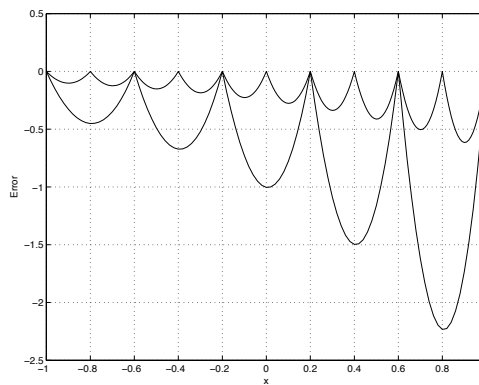
% Plot solution

```

```
plot(x,T,'*-');  
xlabel('x');  
ylabel('T');
```

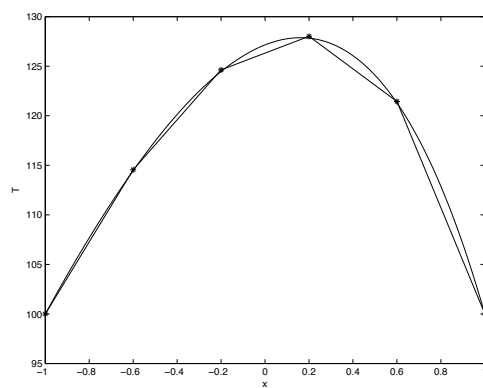
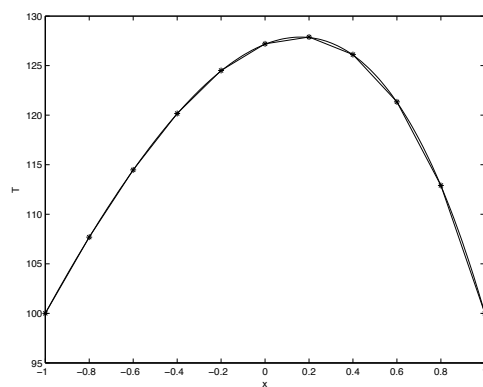
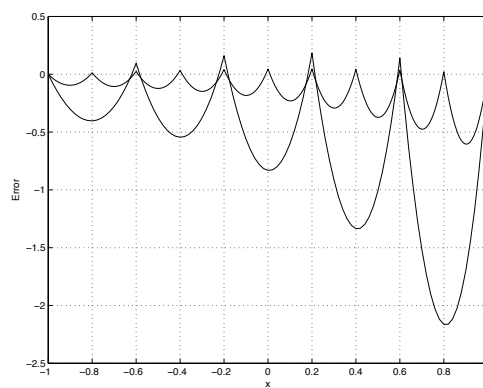
Lecture 12: In-Class Demos and Discussion Topics

- Implementation details of the 1-D FEM in Example 12.2. Matlab code: `fem_dif1d.m`.
- Calculation of forcing integral with Gauss quadrature, in example 12.3. Matlab code: `fem_dif1d_gq.m`.
- Implementation of boundary conditions in `fem_dif1d_all.m`.

(a) $N = 5$ elements(b) $N = 10$ elements

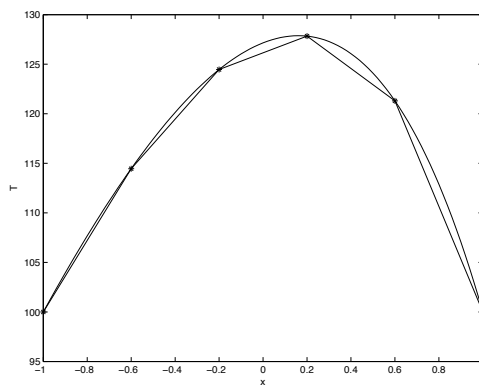
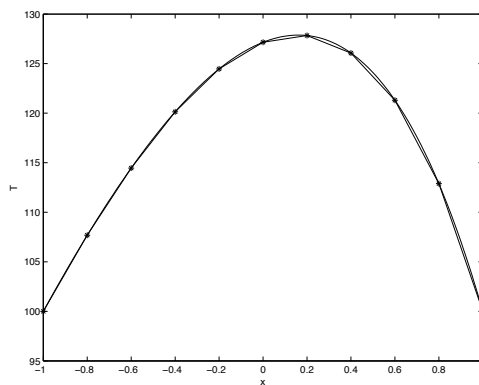
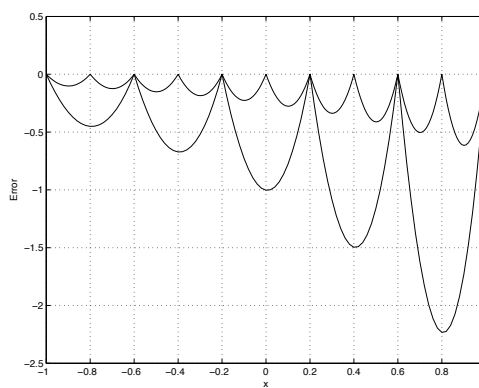
(c) Error

Figure 12.6: Comparison of finite element solution to exact solution.

(a) $N = 5$ elements(b) $N = 10$ elements

(c) Error

Figure 12.7: Comparison of finite element solution using $N_q = 1$ point Gaussian quadrature to exact solution.

(a) $N = 5$ elements(b) $N = 10$ elements

(c) Error

Figure 12.8: Comparison of finite element solution using $N_q = 2$ point Gaussian quadrature to exact solution.

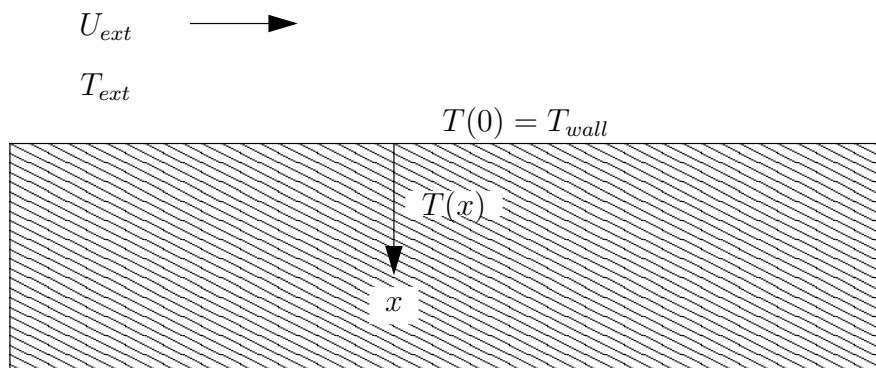


Figure 12.9: Convection above a wall of a flow with velocity U_{ext} and temperature T_{ext} . The temperature of the wall at the surface is T_{wall} (the surface is at $x = 0$).

Lecture 13

The Finite Element Method for Two-Dimensional Diffusion

In this lecture, we will consider the finite element approximation of the two-dimensional diffusion problem,

$$\nabla \cdot (k \nabla T) + f = 0. \quad (13.1)$$

As in the previous discussion of the method of weighted residuals and the finite element method, the approximate solution will have the form,

$$\tilde{T}(x, y) = \sum_{j=1}^N a_j \phi_j(x, y),$$

where $\phi_j(x, y)$ are the known basis functions and the a_j are the unknown coefficients to be determined for the specific problem. Following the Galerkin method of weighted residuals, we will weight Equation (13.1) by one of the basis functions and integrate the diffusion term by parts to give the following weighted residual,

$$R_i \equiv \int_{\delta\Omega} \phi_i k \nabla \tilde{T} \cdot \vec{n} ds - \int_{\Omega} \nabla \phi_i \cdot (k \nabla \tilde{T}) dA + \int_{\Omega} \phi_i f dA = 0. \quad (13.2)$$

13.1 Reference Element and Linear Elements

In multiple dimensions, a common practice in defining the polynomial functions within an element is to transform each element into a canonical, or so-called 'reference' element. Figure 13.1 shows the mapping commonly used for triangular elements which maps a generic triangle in (x, y) into a right triangle in (ξ_1, ξ_2) .

In the reference element space, the nodal basis for linear polynomials will be one at one of the nodes, and reduce linearly to zero at the other nodes. These functions are,

$$\phi_1(\xi_1, \xi_2) = 1 - \xi_1 - \xi_2, \quad (13.3)$$

$$\phi_2(\xi_1, \xi_2) = \xi_1, \quad (13.4)$$

$$\phi_3(\xi_1, \xi_2) = \xi_2. \quad (13.5)$$

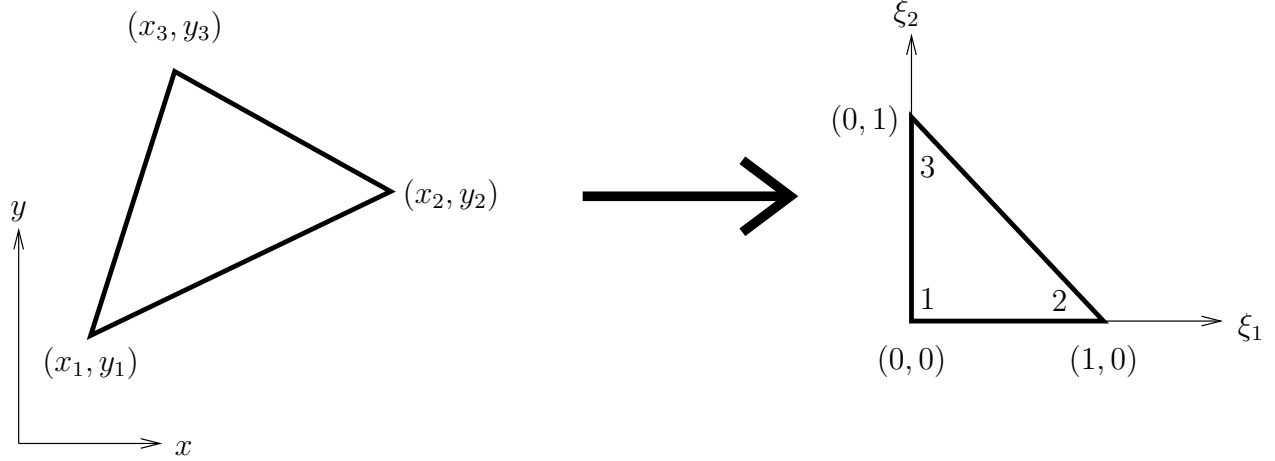


Figure 13.1: Transformation of a generic triangular element in (x, y) into the reference element in (ξ_1, ξ_2) .

Then, within the element, the solution $\tilde{T}$ in the (ξ_1, ξ_2) space is the combination of these three basis functions multiplied by the corresponding nodal coefficients,

$$\tilde{T}(\xi_1, \xi_2) = \sum_{j=1}^3 a_j \phi_j(\xi_1, \xi_2). \quad (13.6)$$

Using Equation (13.6), the value of $\tilde{T}$ can be found at any (ξ_1, ξ_2) . To find the (x, y) locations in terms of the (ξ_1, ξ_2) , we can expand them using the nodal locations and the nodal basis functions, i.e.,

$$\vec{x}(\xi_1, \xi_2) = \sum_{j=1}^3 \vec{x}_j \phi_j(\xi_1, \xi_2).$$

Since the $\phi_i(\xi_1, \xi_2)$ are linear functions of ξ_1 and ξ_2 , this amounts to a linear transformation between (ξ_1, ξ_2) and (x, y) . Specifically, substituting the nodal basis functions gives,

$$\begin{aligned} \vec{x}(\xi_1, \xi_2) &= \vec{x}_1 (1 - \xi_1 - \xi_2) + \vec{x}_2 \xi_1 + \vec{x}_3 \xi_2, \\ \Rightarrow \vec{x}(\xi_1, \xi_2) &= \vec{x}_1 + (\vec{x}_2 - \vec{x}_1) \xi_1 + (\vec{x}_3 - \vec{x}_1) \xi_2. \end{aligned}$$

This can be written in a matrix notation as,

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} + \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix} \begin{pmatrix} \xi_1 \\ \xi_2 \end{pmatrix} \quad (13.7)$$

This equation can be inverted to also determine (ξ_1, ξ_2) as a function of (x, y) ,

$$\begin{pmatrix} \xi_1 \\ \xi_2 \end{pmatrix} = \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix}^{-1} \begin{pmatrix} x - x_1 \\ y - y_1 \end{pmatrix} \quad (13.8)$$

13.2 Differentiation using the Reference Element

To find the derivative of $\tilde{T}$ with respect to x (or similarly y) within an element, we differentiate the three nodal basis functions within the element,

$$\begin{aligned}\tilde{T}_x &= \frac{\partial}{\partial x} \left(\sum_{j=1}^3 a_j \phi_j \right), \\ &= \sum_{j=1}^3 a_j \frac{\partial \phi_j}{\partial x}.\end{aligned}$$

To find the x -derivatives of each of the ϕ_j 's, the chain rule is applied,

$$\frac{\partial \phi_j}{\partial x} = \frac{\partial \phi_j}{\partial \xi_1} \frac{\partial \xi_1}{\partial x} + \frac{\partial \phi_j}{\partial \xi_2} \frac{\partial \xi_2}{\partial x}.$$

Similarly, to find the y -derivatives, the derivatives with respect to y ,

$$\frac{\partial \phi_j}{\partial y} = \frac{\partial \phi_j}{\partial \xi_1} \frac{\partial \xi_1}{\partial y} + \frac{\partial \phi_j}{\partial \xi_2} \frac{\partial \xi_2}{\partial y}.$$

The calculation of the derivatives of ϕ_j with respect to the ξ 's gives,

$$\begin{aligned}\frac{\partial \phi_1}{\partial \xi_1} &= -1, & \frac{\partial \phi_1}{\partial \xi_2} &= -1, \\ \frac{\partial \phi_2}{\partial \xi_1} &= 1, & \frac{\partial \phi_2}{\partial \xi_2} &= 0, \\ \frac{\partial \phi_3}{\partial \xi_1} &= 0, & \frac{\partial \phi_3}{\partial \xi_2} &= 1.\end{aligned}$$

The only remaining terms are the calculation of $\frac{\partial \xi_1}{\partial x}$, $\frac{\partial \xi_2}{\partial x}$, etc. which can be found by differentiating Equation (13.8),

$$\begin{aligned}\frac{\partial \vec{\xi}}{\partial \vec{x}} &= \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix}^{-1}, \\ &= \frac{1}{J} \begin{pmatrix} y_3 - y_1 & -(x_3 - x_1) \\ -(y_2 - y_1) & x_2 - x_1 \end{pmatrix},\end{aligned}$$

where

$$J = (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1).$$

Note that the Jacobian, J , is equal to twice the area of the triangular element.

13.3 Construction of the Stiffness Matrix

The stiffness matrix arises in the calculation of $\int_{\Omega} \nabla \phi_i \cdot (k \nabla \tilde{T}) dA$. As in the one-dimensional case, the i -th row of the stiffness matrix K corresponds to the weighted residual of ϕ_i . The

j -th column in the i -th row corresponds to the dependence of the i -th weighted residual on a_j . Further drawing on the one-dimensional example, the weighted residuals are assembled by calculating the contribution to all of the residuals from within a single element. In the two-dimensional linear element situation, three weighted residuals are impacted by a given element, specifically, the weighted residuals corresponding to the nodal basis functions of the three nodes of the triangle. For example, in each element we must calculate,

$$\int_{\Omega_e} \nabla \phi_1 \cdot (k \nabla \tilde{T}) dA, \quad \int_{\Omega_e} \nabla \phi_2 \cdot (k \nabla \tilde{T}) dA, \quad \int_{\Omega_e} \nabla \phi_3 \cdot (k \nabla \tilde{T}) dA,$$

where Ω_e is spatial domain for a specific element. As described in Section 13.2, the gradient of $\tilde{T}$ can be written,

$$\nabla \tilde{T}(x, y) = \sum_{j=1}^3 a_j \nabla \phi_j(x, y),$$

thus the weighted residuals expand to,

$$\int_{\Omega} \nabla \phi_i \cdot (k \nabla \tilde{T}) dA = \sum_{j=1}^3 a_j K_{i,j}, \quad \text{where} \quad K_{i,j} \equiv \int_{\Omega} \nabla \phi_i \cdot (k \nabla \phi_j) dA.$$

For the situation in which k is constant, and linear elements are used, then this reduces to,

$$K_{i,j} \equiv k \nabla \phi_i \cdot \nabla \phi_j A_e$$

where A_e is the area of element e .

13.4 Integration in the Reference Element

The reference element can also be used to evaluate integrals. For example, consider the evaluation of the forcing function integral within an element,

$$\int_{\delta\Omega_k} w(\vec{x}) f(\vec{x}) dA.$$

In transforming the integral from (x, y) to (ξ_1, ξ_2) , the differential area of integration must be transformed using the following result,

$$dA = dx dy = J d\xi_1 d\xi_2 = J dA_\xi. \quad (13.9)$$

Thus, the integrals can now be evaluated in reference element space,

$$\int_{\Omega_\xi} w(\vec{x}(\vec{\xi})) f(\vec{x}(\vec{\xi})) J dA_\xi.$$

13.5 Calculation of the Mass Matrix

Calculation of the mass matrix will be discussed in class.

Lecture 13: In-Class Demos and Discussion Topics

- Visualization of linear nodal basis functions on triangular meshes.
- Calculation of the mass matrix.
- Integration of the source term using numerical quadrature.

Lecture 14

Higher-order Finite Elements

14.1 Nodal Basis for Higher Order Elements

Until now, we have considered solutions which were allowed to vary at most linearly across an element. We now consider higher-order functions within the element. For simplicity, we will restrict our attention to one-dimensional problems. A p -th order polynomial has $p + 1$ degrees of freedom, i.e. the coefficients of each term,

$$\tilde{T}(x) = c_0 + c_1x + c_2x^2 + \cdots + c_px^p.$$

Thus, a general basis for a p -th order polynomial will require $p + 1$ basis functions within an element,

$$\tilde{T}(x) = \sum_{j=1}^{p+1} a_j \phi_j(x) \quad \text{in an element.}$$

Building on the nodal basis approach described in Section 12.3 for linear elements, a common approach to choosing a basis for higher-order elements is to insert nodes within an element in addition to the nodes at the boundaries of the elements, specifically $p - 1$ additional nodes internal to the element.

Let's consider building a nodal basis for quadratic elements. In this case, one additional node is added and this node is placed at the midpoint of the element. Using the one-dimensional reference element which extends from $-1 \leq \xi \leq 1$, this places nodes at $\xi = -1$, 0 , and 1 . The unknowns are assumed to be the values at these nodes,

$$a_1 = \tilde{T}(-1), \quad a_2 = \tilde{T}(0), \quad a_3 = \tilde{T}(1).$$

which leads to the following constraints on $\phi_i(\xi)$'s,

$$\phi_1(-1) = 1, \quad \phi_1(0) = 0, \quad \phi_1(1) = 0.$$

$$\phi_2(-1) = 0, \quad \phi_2(0) = 1, \quad \phi_2(1) = 0.$$

$$\phi_3(-1) = 0, \quad \phi_3(0) = 0, \quad \phi_3(1) = 1.$$

Applying these constraints and solving for the quadratic $\phi_i(\xi)$ gives,

$$\begin{aligned}\phi_1(\xi) &= -\frac{1}{2}\xi(1-\xi), \\ \phi_2(\xi) &= (1-\xi)(1+\xi), \\ \phi_3(\xi) &= \frac{1}{2}\xi(1+\xi).\end{aligned}$$

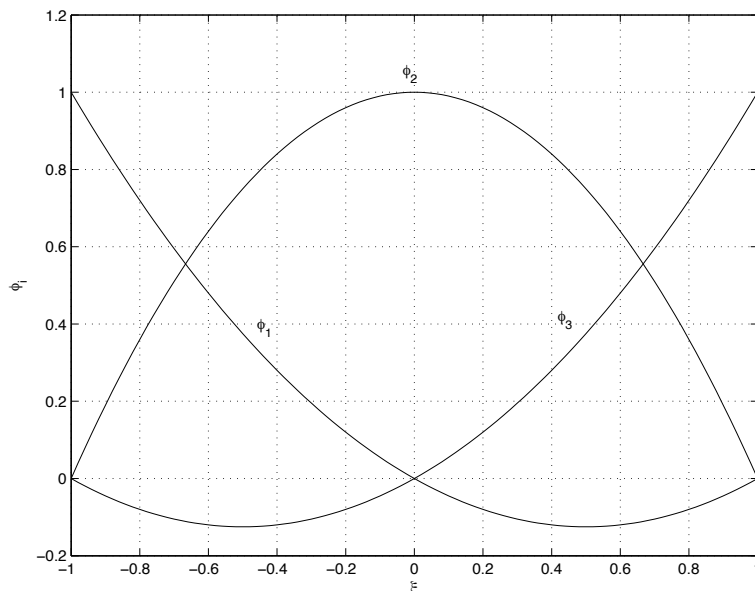


Figure 14.1: Nodal basis functions for a quadratic element with equally-spaced nodes.

14.2 Implementation of higher-order FEM

The implementation details are essentially the same as in the linear element case. Shown below is a Matlab script for a quadratic FEM using a nodal basis.

```
% FEM solver for d2T/dx2 + f = 0 where f = 50 exp(x)
%
% BC's: T(-1) = 100 and T(1) = 100.
%
% Note: the finite element degrees of freedom are
%       stored in the vector, T.
clear all;

% Set number of Gauss points (only used for forcing term in this example)
Nq = 3;
if (Nq == 3),
    xiq = [-sqrt(3/5);    0; sqrt(3/5)];
```

```

    aq = [      5/9; 8/9;      5/9];
elseif (Nq == 2),
    xiq = [-1/sqrt(3); +1/sqrt(3)];
    aq = [      1.0;      1.0];
else,
    Nq = 1;
    xiq = [0.0];
    aq = [2.0];
end

% Number of elements
nElem = 5;
xn = linspace(-1,1,nElem+1);

% Define a local stiffness matrix
Ke = [
    -7/6  4/3  -1/6
     4/3 -8/3   4/3
    -1/6  4/3  -7/6
];

% Zero stiffness matrix and forcing vector
K = zeros(2*nElem+1, 2*nElem+1);
F = zeros(2*nElem+1, 1);

% Loop over all elements and calculate K and F
for elem = 1:nElem,

    % vector of global node numbers on elem
    n = 2*(elem-1) + [1; 2; 3];

    xL = xn(elem);
    xR = xn(elem+1);

    dx    = xR - xL;
    dxidx = 2/dx;
    dxdxi = 1/dxidx;

    % Add to global stiffness matrix
    K(n,n) = K(n,n) + dxidx*Ke;

    % Use Gaussian quadrature to evaluate forcing term integral
    for q = 1:Nq,

        xi = xiq(q); % xi for Gauss point

```

```

% Find phi(1,2,3) (i.e. weighting/intepolants) at xi
phi(1) = -0.5*xi *(1-xi);
phi(2) =  (1-xi)*(1+xi);
phi(3) =  0.5*xi *(1+xi);

% Find x for Gauss point
x = 0.5*(1-xi)*xL + 0.5*(1+xi)*xR;

% Find f for Gauss point
f = -50*exp(x);

% Evaluate integrand at Gauss point for weight functions at nodes
g = phi*f*dx dxi; % note, g is a vector; one entry for each phi

% Send to correct right-hand side term
F(n) = F(n) + aq(q)*g';
end

end

% Set Dirichlet conditions at x=0
n1 = 1;
K(n1,:) = zeros(size(1,2*nElem+1));
K(n1, n1) = 1.0;
F(n1) = 100.0;

% Set Dirichlet conditions at x=1
n1 = 2*nElem+1;
K(n1,:) = zeros(size(1,2*nElem+1));
K(n1, n1) = 1.0;
F(n1) = 100.0;

% Solve for the nodal unknowns, T
T = K\F;

% Plot it and compare. Note: since even the finite element
% solution varies more than linearly across an element, we need to
% subdivide each element, evaluate the basis functions, and plot
% the FEM solution to see the higher order variations.

Nplot = 20; % Number of points per element to plot

```

```

nnn = 0;
for elem = 1:nElem,

    % vector of global node numbers on elem
    n = 2*(elem-1) + [1; 2; 3];

    xL = xn(elem);
    xR = xn(elem+1);

    Te = T(n); % nodal values on elem

    for nn = 1:Nplot,

        % Get xi for plot point
        xi = -1 + 2*(nn-1)/(Nplot-1);

        % Find phi(1,2,3) (i.e. weighting/intepolants) at xi
        phi(1) = -0.5*xi *(1-xi);
        phi(2) =  (1-xi)*(1+xi);
        phi(3) =  0.5*xi *(1+xi);

        % Find x and T for plot point
        x = 0.5*(1-xi)*xL + 0.5*(1+xi)*xR;
        Tplot = dot(Te,phi);

        nnn = nnn + 1;
        xp(nnn) = x;
        Tp(nnn) = Tplot;
        up(nnn) = -50*exp(x) + 50*x*sinh(1) + 100 + 50*cosh(1);

    end
end

figure(1);
plot(xp,Tp,'r');hold on;
plot(xp,up); hold off;
xlabel('x');
ylabel('u');

figure(2);
plot(xp,Tp-up);
xlabel('x');
ylabel('Error');

fprintf(1, 'Maximum error = %g\n', max(abs(Tp-up)));

```

14.3 Hierarchical Basis for Quadratic Elements

In this section, we consider a different basis for quadratic polynomials. Since we want the solution to be continuous from element-to-element, we will still specify that a_1 and a_3 are the values at the end of the elements (i.e. at the nodes),

$$a_1 = \tilde{T}(-1), \quad a_3 = \tilde{T}(1).$$

However, we will no longer associate a_2 with the midpoint value of the temperature. Instead, let the additional constraint be that a_2 is the value of the second derivative in the middle of the reference element, specifically,

$$a_2 = \tilde{T}_{\xi\xi}(0).$$

These three constraints lead to the following conditions on the $\phi_i(\xi)$:

$$\phi_1(-1) = 1, \quad \phi_{1\xi\xi}(0) = 0, \quad \phi_1(1) = 0.$$

$$\phi_2(-1) = 0, \quad \phi_{2\xi\xi}(0) = 1, \quad \phi_2(1) = 0.$$

$$\phi_3(-1) = 0, \quad \phi_{3\xi\xi}(0) = 0, \quad \phi_3(1) = 1.$$

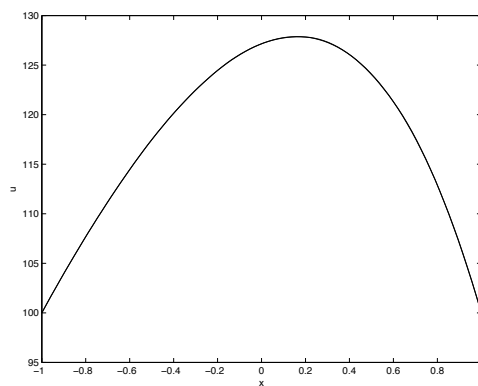
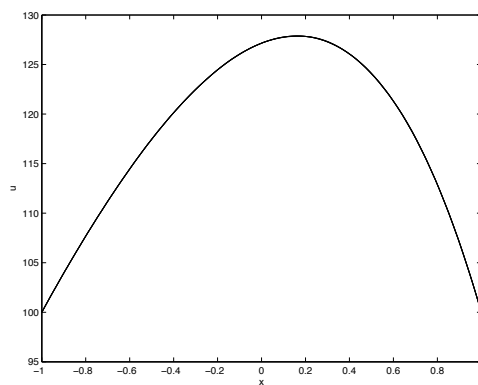
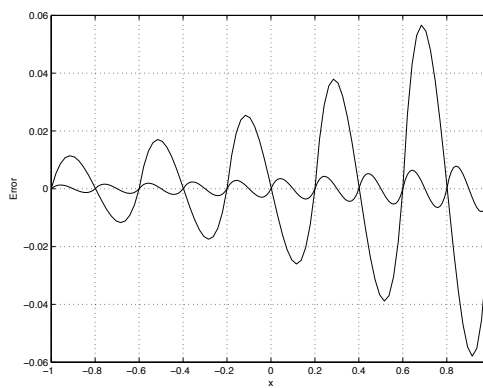
Applying these constraints and solving for the quadratic $\phi_i(\xi)$ gives,

$$\begin{aligned} \phi_1(\xi) &= \frac{1}{2}(1 - \xi), \\ \phi_2(\xi) &= \frac{1}{2}(\xi^2 - 1), \\ \phi_3(\xi) &= \frac{1}{2}(1 + \xi). \end{aligned}$$

This basis is known as a hierarchical basis because the quadratic basis functions are the usual linear basis functions (ϕ_1 and ϕ_3) with an additional function (ϕ_2) that brings the quadratic contribution into the approximate solution. In other words, the basis is hierarchical because the basis for a linear-varying solution is a subset of the basis for the quadratic solution. The plots of ϕ_1 , ϕ_2 , and ϕ_3 are shown in Figure 14.4.

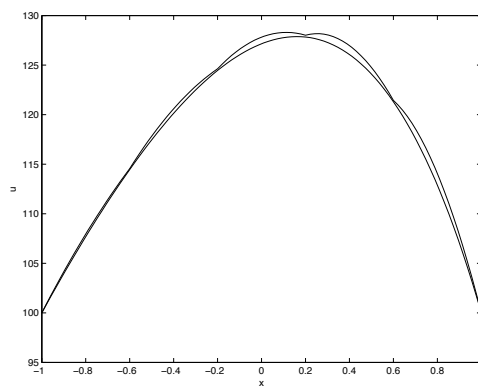
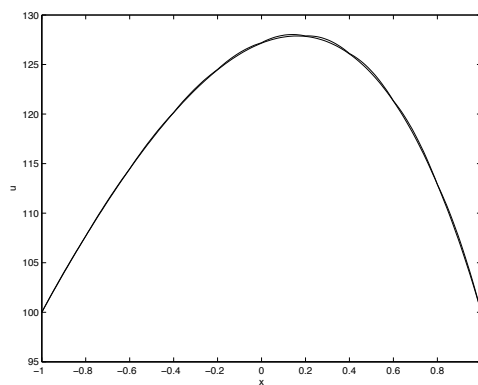
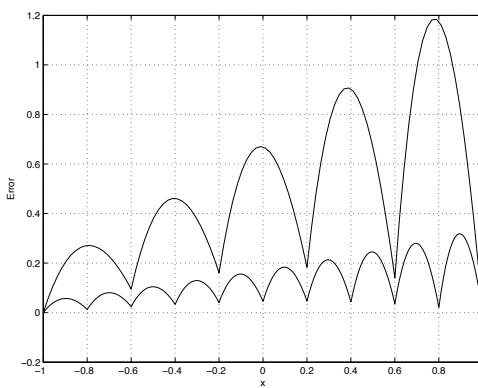
Lecture 14: In-Class Demos and Discussion Topics

- Construction of global $\phi(x)$ for quadratic elements
- Behavior of quadratic FEM: Figures 14.2 and 14.3.
- Rate of convergence for quadratic FEM

(a) $N = 5$ elements(b) $N = 10$ elements

(c) Error

Figure 14.2: Comparison of quadratic finite element solution using 3 point Gaussian quadrature on forcing function.

(a) $N = 5$ elements(b) $N = 10$ elements

(c) Error

Figure 14.3: Comparison of quadratic finite element solution using 1 point Gaussian quadrature on forcing function.

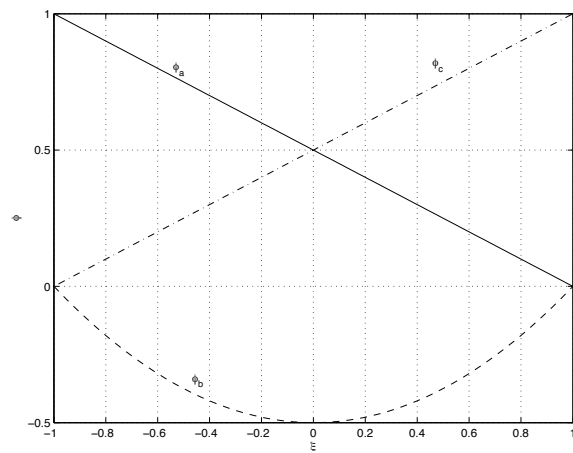
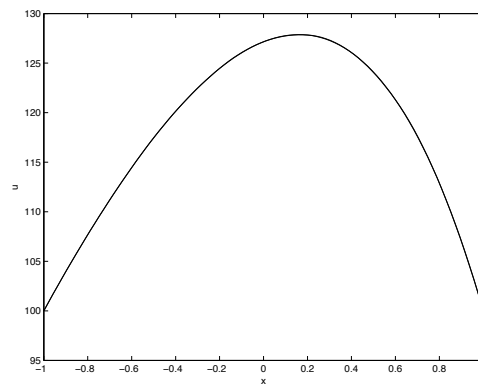
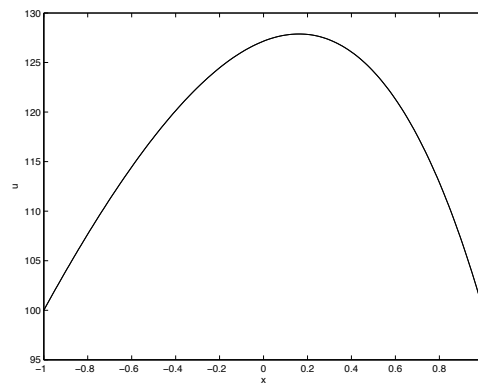
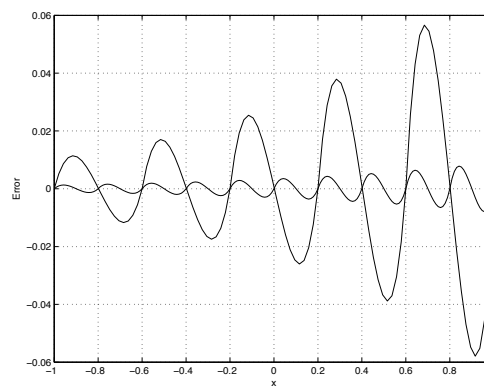


Figure 14.4: Plots of quadratic hierarchical basis functions.

(a) $N = 5$ elements(b) $N = 10$ elements

(c) Error

Figure 14.5: Comparison of quadratic finite element solution with hierarchical basis using 3 point Gaussian quadrature on forcing function.